
Failure-Driven Workflow Refinement

Jusheng Zhang^{*12} Kaitong Cai^{*1} Jing Yang¹ Ziliang Chen¹
Yongsen Zheng² Kwok-Yan Lam² Liang Lin¹ Keze Wang^{†1}

Abstract

Workflow optimization for tool-using LLM agents is often cast as global search over candidate graphs, scored by a scalar metric. This collapses rich, multi-step failure traces into binary outcomes, obscuring recurring failure structure and making refinement inefficient. We reframe optimization as *distributional refinement*: each workflow induces a density over a **Failure Signature Space** $\mathcal{F}$, and the goal is to minimize its **Expected Failure Mass**. We propose **CE-Graph**, which maintains a counterexample pool, estimates dense failure modes, and applies operator-constrained graph edits via a **Propose-and-Verify** loop with a convergence-aware stopping rule. Across math, code, and QA benchmarks, CE-Graph improves robustness while reducing optimization cost compared to strong workflow-search baselines, suggesting reliability emerges from learning and reshaping failure landscapes rather than merely maximizing aggregate success rates.

1. Introduction

Large Language Models (LLMs) (OpenAI et al., 2024; Vaswani et al., 2023; Zhao et al., 2025; Minaee et al., 2025) are increasingly used to power agentic workflows, where tasks are decomposed into multiple steps involving tool calls, logical control flows, and verification routines (Zhang et al., 2025e;a;f; Jeyakumar et al., 2024; Zhang et al., 2025c). Such workflows enable LLMs to handle long-horizon reasoning and complex problem-solving. The central challenge is how to optimize these workflows: given a dataset D , how can we construct a workflow W that achieves the highest reliability?

This challenge is fundamentally distinct from traditional

program synthesis (Yu et al., 2025b; Sapkota et al., 2026). Unlike programs in a formal language, LLM-based workflows operate in a vast, unstructured space defined by natural language prompts and arbitrary tool interactions. Furthermore, their failures are not deterministic bugs but are often **stochastic and semantic** in nature, stemming from subtle flaws in the model’s reasoning process (Steenhoek et al., 2025; Song et al., 2025; Huang et al., 2025; Zhang et al., 2025d;b). Consequently, established methods designed for formal program spaces are often ineffective, necessitating a new optimization paradigm.

A common formulation treats workflow optimization as a global program search (Zhang et al., 2025f;a; Li et al., 2024; Yu et al., 2025a). The objective is to find a configuration W^* that maximizes a global, sample-based evaluation metric $G(W, D)$: $W^* = \arg \max_{W \in \mathcal{S}} G(W, D)$. Recent systems exemplify this view by employing advanced search strategies such as Monte Carlo Tree Search (MCTS) (Coulom, 2006; Browne et al., 2012; undefinedwiechowski et al., 2022). While effective, this line of work inherits a core limitation rooted in its objective function, i.e., the reliance on **information collapse**. The rich, multi-step trace of each failure is compressed into a single binary signal, preventing the optimizer from modeling the **underlying failure distribution**, which is key to efficient refinement.

We argue that these failures are not random noise but are, in fact, samples from a workflow-specific failure distribution (Pan et al., 2025; Zhang et al., 2025g). This insight motivates a paradigm shift. We re-conceptualize workflow optimization not as maximizing a scalar score, but as a process of progressively reshaping this underlying failure distribution. We formalize this by introducing the **Failure Signature Space** $\mathcal{F}$ and casting the optimization goal as the minimization of the **Expected Failure Mass** $M(W)$: $W^* = \arg \min_{W \in \mathcal{S}} M(W)$ where $M(W) = \int_{\mathcal{F}} p(s | W) ds$. This new formulation, which aims to minimize the integral of a failure density function, marks a fundamental departure from the conventional paradigm of maximizing a sample-based performance metric. Instead of treating the workflow as a black box, it provides a white-box and distributional lens. This allows us to move from inefficient, zero-order optimization to a principled, gradient-like descent directly

¹Sun Yat-sen University, Guangzhou, China ²Nanyang Technological University, Singapore. Correspondence to: Keze Wang <keze.wang@sysu.edu.cn>, Jusheng Zhang <jushengzhang1@gmail.com>.

on the landscape of failure modes. To operationalize this principle, we propose CE-Graph (Counterexample-Guided Workflow Optimization), a framework that realizes this failure-mass-reduction strategy. It operates through three key stages. First, it maintains a counterexample pool and systematically diagnoses the densest regions of the failure distribution by clustering their **semantic signatures**, thereby approximating the “semantic gradient”. Second, it replaces heuristic fixes with a principled **Propose-and-Verify** mechanism, which generates and empirically validates targeted graph edits designed to reduce the mass of these failure modes. Third, our CE-Graph incorporates a convergence-aware stopping rule to halt optimization once the failure distribution has stabilized.

We evaluate our CE-Graph on math, code, and QA benchmarks, showing that it achieves higher success rates at significantly lower optimization cost compared to strong workflow search baselines. Beyond these empirical gains, our work highlights a fundamental principle for building robust agentic systems: true reliability emerges not from attempting to avoid failures, but from systematically understanding and resolving their underlying distributional structure. Conceptually, CE-Graph establishes a **self-referential optimization paradigm**, employing language models to analyze and refine the failure distributions of the very LLM-based systems they compose.

2. A New Optimization Paradigm for LLM Workflows

We develop the theoretical foundation of CE-Graph by proposing a distributional paradigm for LLM workflow optimization. We introduce the **Failure Signature Space** and formulate optimization as minimizing the *expected failure mass* in this space (Ribeiro et al., 2020). Unlike scalar success metrics, this view preserves *where/how* failures occur, enabling mode-targeted refinement. We then formalize the dominant “global search” paradigm and show its intrinsic limitation: it observes only collapsed signals and is therefore blind to the geometry of failure distributions (Zhang et al., 2025f). This motivates our paradigm, **failure-driven refinement**, which iteratively reduces failure mass via mode-conditioned edits, setting the stage for the algorithm in Section 3.

2.1. Problem Formulation in Failure Signature Space

Execution traces of LLM workflows are high-dimensional and hard to compare directly. We map failures into a shared, structured space for systematic optimization. Let a workflow be $W \in \mathcal{S}$, and let $\tau \sim \mathcal{D}_W$ denote an execution trace induced by running W . Define a failure indicator $Z(\tau) \in \{0, 1\}$ and a signature mapping $\phi: \mathbf{s} = \phi(\tau) \in \mathcal{F}$, where $\mathcal{F}$ is the **Failure Signature Space**.

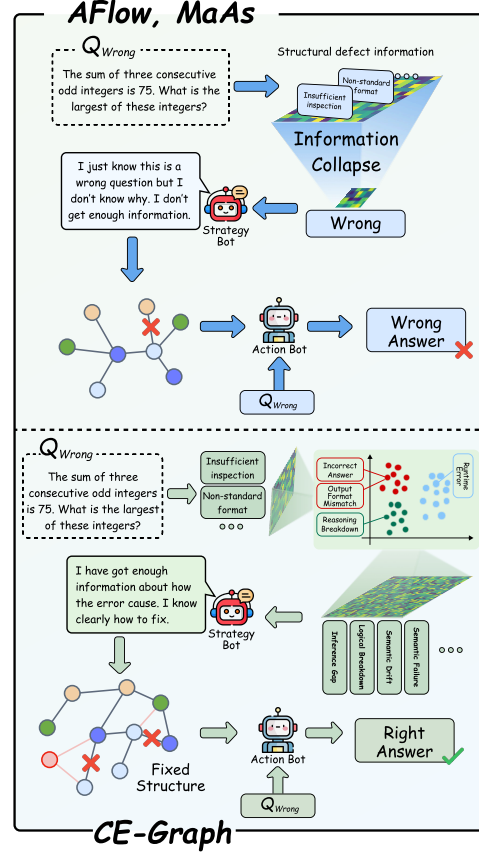


Figure 1. Upper: scalar-only optimization causes “information collapse.” Lower: **CE-Graph** organizes failures in signature space and refines workflows by dominant modes.

Unnormalized failure density. A normalized conditional density over failures would integrate to 1 and cannot directly serve as an objective. We therefore define an *unnormalized* failure density:

$$\rho(\mathbf{s} | W) := \mathbb{P}(Z(\tau) = 1, \phi(\tau) \in d\mathbf{s} | W), \quad (1)$$

which represents the failure-probability mass assigned to regions of $\mathcal{F}$.

Severity-weighted expected failure mass. To account for heterogeneous failure severities, let $\omega(\tau) \geq 0$ be a severity weight (default $\omega \equiv 1$). We define:

$$W^* = \arg \min_{W \in \mathcal{S}} M(W), \quad M(W) = \mathbb{E}_{\tau \sim \mathcal{D}_W} [Z(\tau) \cdot \omega(\tau)] = \int_{\mathcal{F}} \bar{\omega}(\mathbf{s}) \rho(\mathbf{s} | W) d\mathbf{s} \quad (2)$$

where $\bar{\omega}(\mathbf{s}) := \mathbb{E}[\omega(\tau) | \phi(\tau) = \mathbf{s}, Z(\tau) = 1]$. When $\omega \equiv 1$, $M(W)$ reduces to failure probability, while $\rho(\mathbf{s} | W)$ still exposes failure *modes* in $\mathcal{F}$.

We assume (i) traces are i.i.d. samples from $\mathcal{D}_W$ for fixed W , (ii) ϕ captures relevant semantic/structural information,

and (iii) dominant modes of $\rho(s | W)$ are estimable from counterexamples. Edits are discrete and the objective is non-convex; thus we use density-based signals to guide *mode-conditioned refinement* rather than claiming differentiable gradient descent.

2.2. The Pitfalls of Global Search

The dominant global search paradigm (e.g., MCTS in MaAS (Zhang et al., 2025a) / AFlow (Zhang et al., 2025f)) maximizes a black-box score $G(W, D)$ (approximately success rate on dataset D), i.e., $W^* = \arg \max_{W \in S} G(W, D)$. This suffers from **information collapse**: each trace is reduced to $\text{Observe}(\tau_d) \in \{0, 1\}$, discarding *where/why* it failed. In our formulation, the optimizer does not observe $s = \phi(\tau_d)$ and thus cannot model the geometry of $\rho(s | W)$ or isolate dominant failure modes, resulting in inefficient zero-order exploration (Snoek et al., 2012). Our paradigm instead explicitly discovers modes and targets them.

2.3. The Failure-Driven Refinement Paradigm

We reframe optimization as iterative refinement: $W_{t+1} = W_t \oplus \Delta_t$, inspired by counterexample-guided ideas but adapted to stochastic, semantic LLM failures (Debbi, 2018; Shorinwa et al., 2025). Since global minimization is intractable, we adopt a greedy objective:

$$\begin{aligned} \Delta_t &= \arg \max_{\Delta \in \mathcal{A}(W_t, \mathcal{O})} [M(W_t) - M(W_t \oplus \Delta)] \\ &= \arg \max_{\Delta \in \mathcal{A}(W_t, \mathcal{O})} \left[\int_{\mathcal{F}} \bar{\omega}(s) \rho(s | W_t) ds - \int_{\mathcal{F}} \bar{\omega}(s) \rho(s | W_t \oplus \Delta) ds \right] \end{aligned} \quad (3)$$

This is a blueprint for approximation: we target dominant failure modes and apply operator-constrained discrete edits.

2.3.1. THEORETICAL PROPERTIES OF GREEDY MASS REDUCTION

Assume $\rho(s | W)$ is Lipschitz with constant L and edits have bounded impact $\|\rho(s | W \oplus \Delta) - \rho(s | W)\|_\infty \leq B$. Let $b_t^* \subseteq \mathcal{F}$ be the targeted mode.

Theorem 2.1 (Greedy Reduction Bound). *If the edit reduces the (severity-weighted) mass inside b_t^* by at least $\delta > 0$, then $M(W_{t+1}) \leq M(W_t) - \delta + \epsilon$, where $\epsilon = O(L \cdot B \cdot \mu(\mathcal{F} \setminus b_t^*))$ bounds spillover to non-target regions.*

Proof Sketch. Split the integral in Eq. 3 into b_t^* and its complement. The targeted reduction gives $-\delta$ on b_t^* , and Lipschitz plus bounded edit impact yields the spillover term ϵ . Full details are in Appendix G. $\square$

Algorithm 1 The CE-Graph Algorithm

- 1: **Input:** Initial workflow W_0 , Dataset D , Hyperparameters $N, K, k, \epsilon, T_{\max}$
- 2: **Output:** Optimized workflow W_T
- 3: Initialize $t \leftarrow 0$, validation scores $\mathcal{G} \leftarrow \emptyset$
- 4: **while** not converged **and** $t < T_{\max}$ **do**
- 5: *// Sample failures induced by W_t and build an empirical estimate of $\rho(s | W_t)$*
- 6: Populate Counterexample Pool C_t by executing W_t on D .
- 7: **if** C_t is empty **then**
- 8: **break** *// No failures; optimization complete*
- 9: **end if**
- 10: Construct embeddings $S_t = \{\phi(\tau_d) \mid (d, \tau_d) \in C_t\}$.
- 11: Optionally compute severity weights $\Omega_t = \{\omega(\tau_d) \mid (d, \tau_d) \in C_t\}$ (default: $\omega \equiv 1$).
 // Approximate greedy mass reduction (Eq. 3)
- 12: **// Step 1: Target a dominant failure mode**
- 13: Fit GMM to S_t to obtain modes B_t and parameters θ .
- 14: Identify the most consequential mode b_t^* using Eq. 5.
- 15: **// Step 2: Find an effective operator-constrained edit**
- 16: Propose candidates $\{\Delta_1, \dots, \Delta_N\} \subset \mathcal{A}(W_t, \mathcal{O})$ conditioned on b_t^* .
- 17: Select $\Delta_t \leftarrow \arg \max_{\Delta_i} V(\Delta_i)$ via Monte Carlo verification.
- 18: Apply: $W_{t+1} \leftarrow W_t \oplus \Delta_t$.
 // Evaluation and check
- 19: Evaluate g_t on validation set; update $\mathcal{G}$.
- 20: $t \leftarrow t + 1$
- 21: **end while**
- 22: **return** W_t

3. CE-Graph: A Practical Algorithm for Failure Mass Reduction

We detail our CE-Graph as a practical approximation to the intractable greedy failure mass reduction objective (Eq. 3) (Clarke et al., 2000), transforming abstract concepts into actionable steps. By deconstructing the challenge into three principled stages—**(i) empirical failure modeling in signature space**, **(ii) dominant-mode targeting**, and **(iii) propose-and-verify operator-constrained edits**—we operationalize a *mode-conditioned discrete refinement* on the failure landscape (Bishop, 2006). This not only approximates the theoretical ideal but also addresses real-world challenges such as stochastic failures and large edit spaces (Zhang et al., 2025f). The following subsections elaborate on each stage’s role, including a detailed analysis of their approximations and alignments with the theory.

3.1. Constructing the Failure Signature Space $\mathcal{F}$

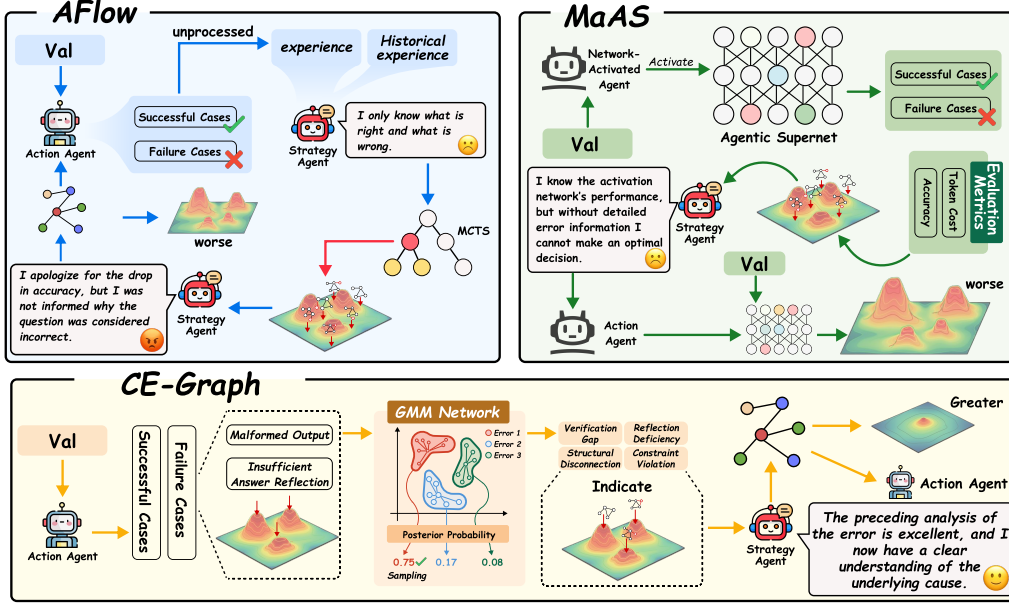


Figure 2. **Overview of CE-Graph.** (i) distill failure traces into signatures; (ii) identify dominant failure modes; (iii) propose-and-verify graph edits to reduce failure mass.

The first step transforms unstructured data in the Counterexample Pool C_t into a tractable format, enabling empirical modeling of $\rho(s | W_t)$. Without this, failures remain opaque, as in global search methods. We define a mapping ϕ that projects a raw trace τ_d into the structured **Failure Signature Space** $\mathcal{F}$ through two stages, ensuring both structural and semantic fidelity.

First, **Failure Distillation** uses a utility LLM to distill τ_d into a concise tuple $(v_{\text{err}}, z_{\text{err}})$, capturing the error-producing node and message. This step compresses verbose traces (e.g., a multi-step math failure involving incorrect LCM calculation) into analyzable components. Second, **Semantic-Structural Vectorization** maps this tuple to the signature vector s , preserving orthogonality and similarity.

Formally, let V be the set of workflow graph nodes and $\mathcal{Z}$ the space of error messages. We define: - A **structural mapping** $\psi_{\text{struct}} : V \rightarrow \mathbb{R}^{|V|}$, yielding a one-hot vector for a node identifier. - A **semantic mapping** $\psi_{\text{sem}} : \mathcal{Z} \rightarrow \mathbb{R}^d$, embedding an error message into d dimensions (e.g., using BERT-like models for semantic clustering).

The final failure signature is:

Definition 3.1 (Failure Signature Vector). A failure signature $s \in \mathcal{F}$ for a distilled trace $(v_{\text{err}}, z_{\text{err}})$ is:

$$s = \phi(\tau_d) = \psi_{\text{struct}}(v_{\text{err}}) \oplus \psi_{\text{sem}}(z_{\text{err}}), \quad (4)$$

where $\oplus$ denotes vector concatenation.

The structural component $\psi_{\text{struct}}(v_{\text{err}})$ identifies *where* the failure occurred, mapping to orthogonal subspaces in $\mathcal{F}$ for node-specific patterns, crucial for attributing blame in

multi-agent workflows. The semantic component $\psi_{\text{sem}}(z_{\text{err}})$ captures *what* error occurred, grouping semantically similar messages (e.g., “calculation error” vs. “the sum was incorrect”). This hybrid embedding endows the point cloud $S_t = \{\phi(\tau_d) \mid \tau_d \in C_t\}$ with an informative geometry for approximating dominant modes of $\rho(s | W_t)$, reducing information loss and exposing failure clusters that global methods miss.

3.2. Solving the Refinement Objective

CE-Graph approximates the greedy mass reduction by two steps: (i) targeting a dominant failure mode, then (ii) finding an effective edit. This decomposition balances computational tractability with theoretical fidelity, ensuring each iteration aims to reduce $M(W_t)$ as per Theorem 2.1.

3.2.1. STEP 1: TARGETING A DOMINANT FAILURE MODE

We approximate the greedy objective by identifying a dominant region of the empirical failure landscape—a mode that contributes the most to the current failure mass. Concretely, we fit a Gaussian Mixture Model (GMM) with parameters θ to the signature set S_t . Each component b_k represents a distinct failure mode (e.g., a cluster of division-by-zero errors in code tasks), with prevalence π_k . To align with the (possibly severity-weighted) objective in Eq. 2, we select the mode with the largest *estimated mass contribution*:

$$b_t^* = \arg \max_{b_k \in B_t} \hat{m}_k, \quad \hat{m}_k := \sum_{s_i \in S_t} p(b_k | s_i, \theta) \cdot \omega(\tau_i), \quad (5)$$

where $p(b_k | s_i, \theta)$ is the posterior responsibility and $\omega(\tau_i)$ is the (optional) severity weight of the corresponding counterexample trace. When $\omega \equiv 1$, this reduces to selecting the most prevalent mode (largest effective π_k), recovering the standard “densest cluster” criterion.

Intuitively, b_t^* serves as a **mode-conditioned direction**: it identifies a systematic weakness to prioritize, so that subsequent edits address concentrated failure mass rather than isolated outliers.

3.2.2. ANALYSIS OF MODE TARGETING APPROXIMATION

The GMM provides a consistent estimator of the dominant structure in the empirical signature distribution under standard conditions, making it a practical proxy for the true (unnormalized) failure density $\rho(s | W_t)$. As $|S_t|$ grows, the estimated responsibilities concentrate around coherent modes, and the selected b_t^* increasingly reflects the region with the largest contribution to $M(W_t)$. Importantly, this step is *not* a differentiable gradient computation: workflow edits are discrete, and our goal is to obtain a high-impact target region for refinement. In benchmarks like MATH (Hendrycks et al., 2021), where failures cluster around recurring algebraic misconceptions, mode targeting effectively identifies “mountains” whose reduction yields outsized improvements, compared to uniform sampling that spreads budget across low-impact regions.

3.2.3. STEP 2: FINDING THE OPTIMAL EDIT (Δ_t) VIA PROPOSE-AND-VERIFY

This step addresses two challenges: the vast edit space $\mathcal{A}(W_t, \mathcal{O})$ (potentially exponential in workflow size) and the unknown utility distribution over instances. Our **Propose-and-Verify** mechanism handles them in two stages, combining LLM creativity with empirical validation to approximate the argmax in Eq. 3.

Constrained Proposal: A “Proposer” LLM, conditioned on b_t^* ’s summary (e.g., “recurring off-by-one errors in loops”), generates N diverse high-potential edits $\{\Delta_1, \dots, \Delta_N\}$, reducing the search to a promising subset. This heuristic leverages LLM’s generative power while constraining outputs to the operator library $\mathcal{O}$, ensuring feasibility and keeping the refinement step local.

Verification: To align with mass reduction, we evaluate each candidate on K sampled instances associated with the target mode b_t^* and estimate its post-edit failure mass on that mode. Let $\tau_k^{(i)}$ be the trace obtained by executing $W_t \oplus \Delta_i$ on sampled instance x_k (from the counterexamples assigned to b_t^*), and let $Z(\tau_k^{(i)})$ indicate failure under the verifier. We

define:

$$V(\Delta_i) \approx -\frac{1}{K} \sum_{k=1}^K Z(\tau_k^{(i)}) \cdot \omega(\tau_k^{(i)}), \quad (6)$$

so that larger $V(\Delta_i)$ corresponds to smaller estimated failure mass on the target mode. The highest $V(\Delta_i)$ yields Δ_t , enabling efficient, evidence-based refinement. This Monte Carlo estimator directly tracks the quantity our theory aims to reduce (Eq. 2), with variance decreasing as $O(1/K)$ under bounded ω , aligning with the spirit of Theorem 2.1. Further details are provided in Appendix H.

4. Experiments

We benchmark our *failure-driven refinement* paradigm against a comprehensive suite of baselines across mathematical reasoning, code generation, and complex tool-use tasks, to evaluate both (i) final-task performance and (ii) optimization efficiency.

4.1. Experimental Setup

Tasks and Benchmarks. We evaluate CE-Graph on public benchmarks spanning three domains: **(1) Math reasoning:** GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), and MultiArith (Roy & Roth, 2015); **(2) Code generation:** HumanEval and MBPP (Chen et al., 2021; Austin et al., 2021); **(3) Tool use:** GAIA (Mialon et al., 2023). For GSM8K/MATH/MultiArith, we report exact-match accuracy; for HumanEval/MBPP, we report `pass@1` under the standard evaluation harness. For GAIA, we follow the official Level 1–3 protocol and report accuracy per level and the overall average. **Baselines.** We compare against three categories: **Single-agent execution** (Vanilla, CoT (Wei et al., 2022), ComplexCoT (Kojima et al., 2022), and Self-Consistency (Wang et al., 2023)); **Hand-crafted multi-agent systems** (MultiPersona (Wang et al., 2024), LLM-Debate (Du et al., 2024), LLM-Blender (Jiang et al., 2023), DyLAN (Liu et al., 2024b), AgentVerse (Chen et al., 2024b), MacNet (Qian et al., 2025)); **Automated agentic systems** (AutoAgents (Chen et al., 2024a), GPTSwarm (Zhuge et al., 2024), ADAS (Hu et al., 2025), AgentSquare (Shang et al., 2025), AFlow (Zhang et al., 2025f), and MaAS (Zhang et al., 2025a)). Unless a baseline defines a mandatory controller, we standardize the execution environment (same tool access when applicable) to ensure comparability.

Base LLM and Decoding. All methods use the same base model, `gpt-4o-mini` (OpenAI et al., 2024), to avoid conflating algorithmic gains with model capacity. We use deterministic decoding (temperature = 0) for single-run methods. For Self-Consistency (CoT×5), we follow the standard setting with 5 independent samples and majority vote. For methods that require multiple agents, each agent is instanti-

Table 1. A complete performance comparison on math and code benchmarks. Results are highlighted using shades of gray for the header and top performers, bold for the best, and underlining for the runner-up.

Method	GSM8K	MATH	MultiArith	HumanEval	MBPP	Avg.
<i>Single Agent Execution</i>						
Vanilla (Baseline)	87.45	46.29	96.85	87.08	71.83	77.50
CoT (Wei et al., 2022)	87.10	46.40	96.31	88.13	71.83	77.95
ComplexCoT (Kojima et al., 2022)	86.89	46.53	96.70	87.49	72.36	78.00
SC (Wang et al., 2023) (CoT $\times$ 5)	87.57	47.91	96.58	88.60	73.60	78.85
<i>Hand-craft Multi-agent Systems</i>						
MultiPersona (Wang et al., 2024)	87.50	45.43	97.49	88.32	73.19	78.39
LLM-Debate (Du et al., 2024)	89.47	48.54	97.33	88.68	70.29	78.86
LLM-Blender (Jiang et al., 2023)	88.35	46.92	97.29	88.80	77.05	79.68
DyLAN (Liu et al., 2024b)	89.98	48.63	97.12	90.42	77.30	80.69
AgentVerse (Chen et al., 2024b)	89.91	47.35	97.50	89.29	74.28	79.67
MacNet (Qian et al., 2025)	87.95	45.18	96.03	84.57	65.28	75.00
<i>Automated Agentic Systems</i>						
AutoAgents (Chen et al., 2024a)	87.69	45.32	96.42	87.64	71.95	77.80
GPTSwarm (Zhuge et al., 2024)	89.14	47.88	96.79	89.32	77.43	80.11
ADAS(Hu et al., 2025)	86.12	43.18	96.02	84.19	68.13	75.13
AgentSquare(Shang et al., 2025)	87.62	48.51	97.77	89.08	78.46	80.29
AFlow(Zhang et al., 2025f)	91.16	51.28	96.22	90.93	81.67	82.25
MaAS(Zhang et al., 2025a)	<u>92.30</u>	51.82	98.80	92.85	<u>82.17</u>	83.59
CE-Graph (Ours)	93.70	55.91	99.20	94.26	88.10	86.23

ated as an independent call to the same base LLM, with role prompts specified by the method. A key concern in agentic optimization is whether gains come from *larger budgets* rather than better algorithms. To ensure strict comparability, we enforce **budget parity** at two levels:

(i) Deployment-time inference budget (per test instance).

Across all methods, we impose the same hard caps on resource usage: a maximum number of LLM calls, a maximum total token budget (prompt+completion), and (when applicable) a maximum number of tool/verifier calls. Multi-sample or multi-agent baselines (e.g., SC, Debate, DyLAN) must allocate within the same global caps, rather than exceeding them via extra agents/samples. We report the *realized* inference-time tokens and costs in our cost analysis (Fig. 3, right), so improvements cannot be attributed to larger per-instance budgets. **(ii) Offline optimization budget (for automated workflow search/refinement).** For automated agentic systems (AutoAgents/GPTSwarm/ADAS/AgentSquare/AFlow/MaAS) and CE-Graph, we additionally control the total optimization budget: the total number of candidate workflow evaluations and the total optimization tokens consumed during search/refinement are fixed to the same cap. Concretely, each method is run under the same maximum number of refinement/search iterations and the same validation protocol, and is stopped early only if it meets its own convergence criterion within the shared budget. Thus, the final workflow used in Table 1 is the best one

found under an equal optimization budget, not an unequal search cost.

CE-Graph Initialization, Optimization Budget, and Operators.

CE-Graph starts from a minimal initial workflow W_0 (a single solve node plus a verifier) and iteratively refines it by counterexample-guided local graph edits. We run optimization for up to 20 iterations and stop early if validation improvement saturates under our convergence-aware criterion (Section 3). Our operator library $\mathcal{O}$ contains three executable edits: `RevisePrompt` (prompt repair for an existing node), `InsertNode` (add a new sub-step with explicit I/O), and `DeleteNode` (remove redundant/degenerate steps). Each proposed edit is validated via a propose-and-verify stage; only edits that improve validation metrics are committed to update W_{t+1} .

4.2. Performance Analyses

As shown in Table 1, CE-Graph consistently outperforms all baselines across every task domain and establishes a new state-of-the-art average score of **86.23%**, widening the gap over the previous best of 83.59% achieved by MaAS. The gains are substantial across categories: on the **MATH** benchmark, CE-Graph improves accuracy by +4.1%; on the **MBPP** code synthesis task, it surpasses MaAS by +5.9%; and on **HumanEval**, it delivers an additional +1.4% improvement. Together, CE-Graph demonstrates balanced

improvements across mathematical reasoning, program synthesis, and tool-use domains. Figure 3 highlights the advantages of our failure-driven paradigm in complex, multi-domain environments like GAIA. Unlike monolithic systems (e.g., AFlow, ADAS) that struggle to cover heterogeneous task requirements with a static workflow, CE-Graph iteratively repairs prevalent failure modes, evolving into a robust workflow capable of cross-domain adaptation. This drives substantial gains across all levels—especially Level 3—underscoring its general-purpose capability. Furthermore, CE-Graph exhibits remarkable consistency across math, code, and tool-use. Crucially, its advantage widens with difficulty: while baselines plateau on long-horizon problems, CE-Graph maintains steady gains by systematically resolving structural weaknesses in the failure landscape.

4.3. Cost Analyses

Beyond raw performance, we analyze both **deployment-time** and **optimization-time** costs. For deployment-time efficiency, Fig. 3 (right) reports the realized token usage and API cost under the same inference budget caps for all methods, highlighting CE-Graph’s favorable accuracy–efficiency trade-off. For optimization-time efficiency, CE-Graph reduces redundant trial-and-error by targeting dominant failure modes and validating only a small set of operator-constrained edits, leading to lower optimization token consumption under an equal search/refinement budget. Moreover, our convergence-aware stopping criterion prevents unnecessary iterations once validation improvements saturate, yielding substantial savings compared to fixed-iteration optimization. Ablations in Section 4.5 further confirm that clustering, verification, and structured operators are all critical for achieving strong performance *without* increasing budget.

4.4. Clustering-Driven Correction Stability

To precisely evaluate the stability and sustained refinement capability of our CE-Graph, we design a longitudinal assessment aimed at verifying its long-term effectiveness in locating and resolving fundamental failures. We begin by executing an unoptimized baseline workflow (W_0) on three mathematical reasoning benchmarks, i.e., GSM8K, MATH, and MultiArith, and collect all resulting failure cases to construct a fixed initial failure set (E_0). During the subsequent 20 rounds of optimization, each updated workflow (W_t) is re-evaluated on this fixed set (E_0). We continuously track a single core metric: *Accuracy on E_0* , defined as the proportion of initial failure examples successfully repaired by W_t . This fixed-set evaluation design effectively isolates variables, thereby providing a clear lens through which to assess the refinement mechanism’s inherent capacity for stable and accumulative improvements over time. The lon-

Table 2. Ablation of CE-Graph on MATH and HumanEval. We introduce w/ *Generic Operators* to verify automation capabilities without domain-specific engineering. Metrics: Acc./pass@1 (%); Cost: optimization tokens ($\times 10^3$).

Method	MATH		HumanEval	
	Acc.	Cost	pass@1	Cost
CE-Graph (Full)	55.91	1210	94.26	955
w/o Clustering	51.25	1190	91.50	940
w/o Verification	49.10	1250	89.75	980
w/ Generic Operators	53.40	1290	92.10	1020
w/o Structured Operators	47.35	1310	87.20	1050
w/o Conv.-aware Stopping	53.50	1850	93.80	1520

gitudinal evaluation conducted on fixed failure sets (E_0) across the three mathematical reasoning benchmarks clearly demonstrates the superior stability and sustained improvement capability of our CE-Graph. As shown in Figure 4, CE-Graph achieves a smooth, monotonic trajectory, avoiding the *policy oscillation* and *information collapse* typical of baselines like AFlow. Through failure clustering and principled verification, it transforms discrete signals into consistent, net-positive iterations.

4.5. Ablation Studies

To validate CE-Graph’s components and automation capability, we evaluate five variants: (1) **w/o Clustering**: excludes failure pattern identification; (2) **w/o Verification**: omits the validation step; (3) **w/ Generic Operators**: restricts actions to fundamental edits (e.g., *Add*, *Modify*) without domain priors to test efficacy under minimal human intervention; (4) **w/o Structured Operators**: replaces the library with free-form editing; and (5) **w/o Conv.-aware Stopping**: removes adaptive stopping. We analyze performance (Acc., pass@1, cost) and optimization dynamics on MATH and HumanEval to isolate each module’s impact.

Table 2 validates CE-Graph’s component synergy. **w/ Generic Operators** maintains competitive performance (MATH: 53.40%, HumanEval: 92.10%), proving robust automation even without domain priors. In contrast, the sharp drop in **w/o Structured Operators** (MATH: 47.35%) confirms the necessity of discrete action spaces for executable repairs. **w/o Verification** and **w/o Clustering** highlight the critical roles of hallucination filtering and mode-targeting, respectively. Finally, **w/o Convergence-aware Stopping** spikes costs by $> 50\%$, demonstrating its essential role in efficiency.

5. Related Works

Automated Optimization of Agentic Workflows Recent systems (e.g., MaAS (Zhang et al., 2025a), AFlow (Zhang et al., 2025f), AgentSquare (Shang et al., 2025), AdaptFlow (Zhu et al., 2025)) optimize agentic workflows via global search (MCTS/evolution) to maximize success rates.

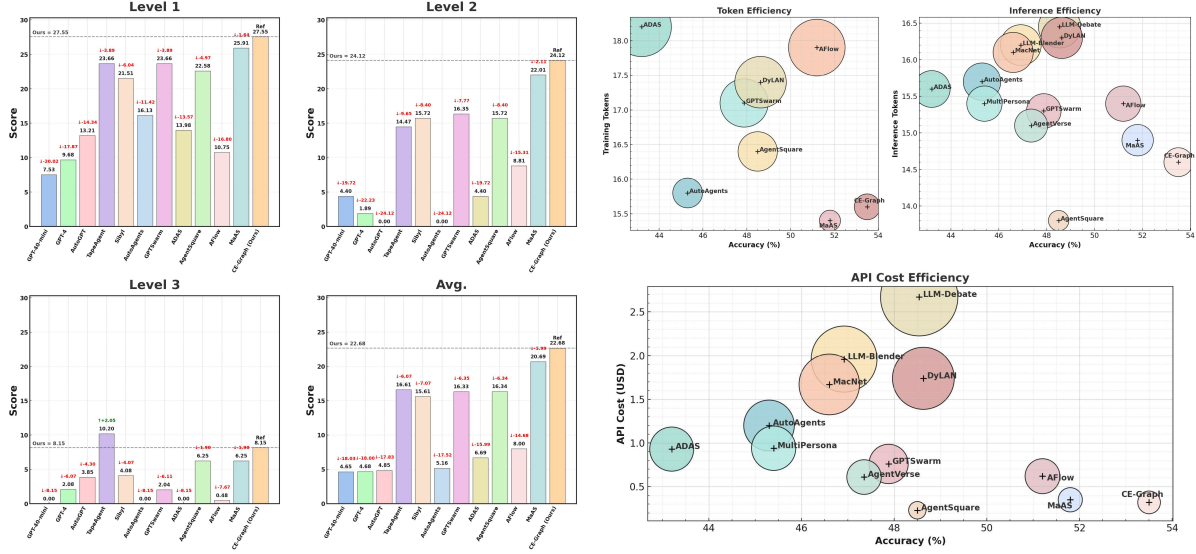


Figure 3. **CE-Graph** outperforms all baselines on GAIA (Levels 1–3, Avg.) and achieves the best accuracy–efficiency trade-off (tokens, API cost) in the ideal lower-right region.

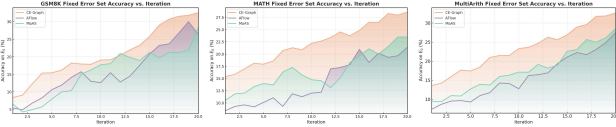


Figure 4. Refinement accuracy on a fixed failure set (E_0) over 20 rounds on GSM8K, MATH, and MultiArith. **CE-Graph** improves monotonically; baselines (e.g., **AFlow**) fluctuate.

Prompt optimizers such as DSPy (Khattab et al., 2024) also rely on scalar rewards, but this can induce “information collapse” by compressing rich failure traces into coarse signals, leading to expensive zero-order exploration. CE-Graph instead preserves semantic and structural failure information in a Failure Signature Space and optimizes via Expected Failure Mass minimization, enabling targeted, cost-effective repairs and improved robustness.

Counterexample-Guided Refinement in Formal Methods Counterexample-guided refinement, central to program synthesis and verification (e.g., CEGAR (Hidvégi et al., 2024), CigaR (Renze & Guven, 2024)), iteratively fixes models using violating examples. However, classical methods are designed for symbolic/deterministic settings and do not directly handle the stochastic, semantic failures of LLM workflows (Chang et al., 2024). CE-Graph adapts the principle to open-ended LLM domains by replacing formal checking with empirical failure-mode clustering and constrained graph edits, improving reliability beyond what symbolic checkers can capture.

Instance-Level Self-Correction and Reflection Instance-level self-correction methods (Self-Consistency (Wang et al., 2023), Reflexion (Shinn et al., 2023), Self-Refine (Madaan et al., 2023)) improve single outputs via voting or reflection, and are studied in agent benchmarks such as AgentBench (Liu et al., 2024a). While effective on individual cases, they often miss systemic failure patterns and can be unstable across iterations. CE-Graph takes a distributional view: it clusters counterexamples into recurring modes and applies structured repairs to fix root causes, yielding more stable and scalable refinement across domains.

6. Conclusion

We select CE-Graph as the optimization framework for agentic workflows because it directly addresses a core limitation of prior approaches: the collapse of rich failure traces into scalar rewards. By modeling failures in a structured signature space and explicitly identifying recurring failure modes, CE-Graph enables targeted, mode-level refinement rather than heuristic or search-heavy adjustments. The Propose-and-Verify mechanism further ensures that each edit is both constrained and empirically validated, yielding stable improvements without incurring the high cost and instability of global search.

Impact Statement

Our work advances the field of automated agent design by introducing a resource-efficient optimization framework, CE-Graph. By focusing on distributional failure analysis rather than brute-force search, we drastically lower the computa-

tional cost required to build robust agentic systems. While this improves the accessibility and reliability of autonomous agents, it also underscores the need for responsible deployment. As agents become more capable of self-correction and complex reasoning, ensuring these capabilities are aligned with human values and safety standards remains a critical priority for the community.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62276283; in part by the China Meteorological Administration’s Science and Technology Project under Grant CMAJBGS202517; in part by the Guangdong-Hong Kong-Macao Greater Bay Area Meteorological Technology Collaborative Research Project under Grant GHMA2024Z04; in part by the Fundamental Research Funds for the Central Universities, Sun Yat-sen University, under Grants 23hytd006 and 23hytd006-2; in part by the Guangdong Provincial High-Level Young Talent Program under Grant RL2024-151-2-11; and in part by the National Research Foundation, Singapore, and Infocomm Media Development Authority under its Trust Tech Funding Initiative. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore, or Infocomm Media Development Authority.

References

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006. ISBN 978-0-387-31073-2. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>.
- Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., and Colton, S. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012. doi: 10.1109/TCIAIG.2012.2186810.
- Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., and Xie, X. A survey on evaluation of large language models. *ACM Trans. Intell. Syst. Technol.*, 15(3), March 2024. ISSN 2157-6904. doi: 10.1145/3641289. URL <https://doi.org/10.1145/3641289>.
- Chen, G., Dong, S., Shu, Y., Zhang, G., Sesay, J., Karlsson, B. F., Fu, J., and Shi, Y. Autoagents: A framework for automatic agent generation, 2024a. URL <https://arxiv.org/abs/2309.17288>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Chan, C.-M., Yu, H., Lu, Y., Hung, Y.-H., Qian, C., Qin, Y., Cong, X., Xie, R., Liu, Z., Sun, M., and Zhou, J. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=EHg5GDnyq1>.
- Clarke, E. M., Grumberg, O., Jha, S., Lu, Y., and Veith, H. Counterexample-guided abstraction refinement. pp. 154–169, 2000. doi: 10.1007/10722167_15. URL https://link.springer.com/chapter/10.1007/10722167_15.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Coulom, R. Efficient selectivity and backup operators in monte-carlo tree search. In *Proceedings of the 5th International Conference on Computers and Games*, pp. 72–83, 2006.
- Debbi, H. Counterexamples in model checking: A survey. *Informatica*, 42(2):145–166, 2018. URL <https://www.informatica.si/index.php/informatica/article/view/1442>.

- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org, 2024.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Hidvégi, D., Etemadi, K., Bobadilla, S., and Monperrus, M. Cigar: Cost-efficient program repair with llms, 2024. URL <https://arxiv.org/abs/2402.06598>.
- Hu, S., Lu, C., and Clune, J. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=t9U3LW7JVX>.
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), January 2025. ISSN 1046-8188. doi: 10.1145/3703155. URL <https://doi.org/10.1145/3703155>.
- Jeyakumar, S. K., Ahmad, A. A., and Gabriel, A. G. Advancing agentic systems: Dynamic task decomposition, tool integration and evaluation using novel metrics and dataset. In *NeurIPS 2024 Workshop on Open-World Agents*, 2024. URL <https://openreview.net/forum?id=kRRLhPp7CO>.
- Jiang, D., Ren, X., and Lin, B. Y. LLM-blender: Ensembling large language models with pairwise ranking and generative fusion. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14165–14178, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.792. URL <https://aclanthology.org/2023.acl-long.792/>.
- Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. Dspy: Compiling declarative language model calls into self-improving pipelines. 2024.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- Li, Z., Xu, S., Mei, K., Hua, W., Rama, B., Raheja, O., Wang, H., Zhu, H., and Zhang, Y. Autoflow: Automated workflow generation for large language model agents, 2024. URL <https://arxiv.org/abs/2407.12821>.
- Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., and Tang, J. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Liu, Z., Zhang, Y., Li, P., Liu, Y., and Yang, D. Dynamic LLM-agent network: An LLM-agent collaboration framework with agent team optimization, 2024b. URL <https://openreview.net/forum?id=i43XCU54Br>.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoy, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. Self-refine: Iterative refinement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=S37hOerQLB>.
- Mialon, G., Fourier, C., Swift, C., Wolf, T., LeCun, Y., and Scialom, T. Gaia: a benchmark for general ai assistants, 2023. URL <https://arxiv.org/abs/2311.12983>.
- Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., and Gao, J. Large language models: A survey, 2025. URL <https://arxiv.org/abs/2402.06196>.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., and Button, K. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- Pan, M. Z., Cemri, M., Agrawal, L. A., Yang, S., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Ramchandran, K., Klein, D., Gonzalez, J. E., Zaharia, M., and Stoica, I. Why do multiagent systems fail? In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025. URL <https://openreview.net/forum?id=wM521FqPvI>.

- Qian, C., Xie, Z., Wang, Y., Liu, W., Zhu, K., Xia, H., Dang, Y., Du, Z., Chen, W., Yang, C., Liu, Z., and Sun, M. Scaling large language model-based multi-agent collaboration. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=K3n5jPkrU6>.
- Renze, M. and Guven, E. The benefits of a concise chain of thought on problem-solving in large language models. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pp. 476–483. IEEE, November 2024. doi: 10.1109/fllm63129.2024.10852493. URL <http://dx.doi.org/10.1109/FLLM63129.2024.10852493>.
- Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. Beyond accuracy: Behavioral testing of NLP models with CheckList. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.442. URL <https://aclanthology.org/2020.acl-main.442/>.
- Roy, S. and Roth, D. Solving general arithmetic word problems. In Márquez, L., Callison-Burch, C., and Su, J. (eds.), *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 1743–1752, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1202. URL <https://aclanthology.org/D15-1202/>.
- Sapkota, R., Roumeliotis, K. I., and Karkee, M. Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. *Information Fusion*, 126:103599, February 2026. ISSN 1566-2535. doi: 10.1016/j.inffus.2025.103599. URL <http://dx.doi.org/10.1016/j.inffus.2025.103599>.
- Shang, Y., Li, Y., Zhao, K., Ma, L., Liu, J., Xu, F., and Li, Y. Agentsquare: Automatic llm agent search in modular design space, 2025. URL <https://arxiv.org/abs/2410.06153>.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. R., and Yao, S. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=vAE1hFcKW6>.
- Shorinwa, O., Mei, Z., Lidard, J., Ren, A. Z., and Majumdar, A. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions. *ACM Comput. Surv.*, 58(3), September 2025. ISSN 0360-0300. doi: 10.1145/3744238. URL <https://doi.org/10.1145/3744238>.
- Snoek, J., Larochelle, H., and Adams, R. P. Practical bayesian optimization of machine learning algorithms, 2012. URL <https://arxiv.org/abs/1206.2944>.
- Song, P., Han, P., and Goodman, N. A survey on large language model reasoning failures. In *2nd AI for Math Workshop @ ICML 2025*, 2025. URL <https://openreview.net/forum?id=hsgMn4KBFg>.
- Steenhoek, B., Rahman, M. M., Roy, M. K., Alam, M. S., Tong, H., Das, S., Barr, E. T., and Le, W. To err is machine: Vulnerability detection challenges llm reasoning, 2025. URL <https://arxiv.org/abs/2403.17218>.
- undefinedwiechowski, M., Godlewski, K., Sawicki, B., and Mańdziuk, J. Monte carlo tree search: a review of recent modifications and applications. *Artif. Intell. Rev.*, 56(3): 2497–2562, July 2022. ISSN 0269-2821. doi: 10.1007/s10462-022-10228-y. URL <https://doi.org/10.1007/s10462-022-10228-y>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Wang, Z., Mao, S., Wu, W., Ge, T., Wei, F., and Ji, H. Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration. In Duh, K., Gomez, H., and Bethard, S. (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 257–279, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.15. URL <https://aclanthology.org/2024.naacl-long.15/>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

- Yu, C., Cheng, Z., Cui, H., Gao, Y., Luo, Z., Wang, Y., Zheng, H., and Zhao, Y. A survey on agent workflow – status and future. In *2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD)*, pp. 770–781, 2025a. doi: 10.1109/ICAIBD64986.2025.11082076.
- Yu, C., Cheng, Z., Cui, H., Gao, Y., Luo, Z., Wang, Y., Zheng, H., and Zhao, Y. A survey on agent workflow – status and future. In *2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD)*, pp. 770–781. IEEE, May 2025b. doi: 10.1109/icaibd64986.2025.11082076. URL <http://dx.doi.org/10.1109/ICAIBD64986.2025.11082076>.
- Zhang, G., Niu, L., Fang, J., Wang, K., BAI, L., and Wang, X. Multi-agent architecture search via agentic supernet. In *Forty-second International Conference on Machine Learning*, 2025a. URL <https://openreview.net/forum?id=imcyVlzpXh>.
- Zhang, J., Cai, K., Fan, Y., Wang, J., and Wang, K. Cf-vlm:counterfactual vision-language fine-tuning, 2025b. URL <https://arxiv.org/abs/2506.17267>.
- Zhang, J., Fan, Y., Cai, K., Sun, X., and Wang, K. Osc: Cognitive orchestration through dynamic knowledge alignment in multi-agent llm collaboration, 2025c. URL <https://arxiv.org/abs/2509.04876>.
- Zhang, J., Fan, Y., Lin, W., Chen, R., Jiang, H., Chai, W., Wang, J., and Wang, K. Gam-agent: Game-theoretic and uncertainty-aware collaboration for complex visual reasoning, 2025d. URL <https://arxiv.org/abs/2505.23399>.
- Zhang, J., Huang, Z., Fan, Y., Liu, N., Li, M., Yang, Z., Yao, J., Wang, J., and Wang, K. KABB: Knowledge-aware bayesian bandits for dynamic expert coordination in multi-agent systems. In *Forty-second International Conference on Machine Learning*, 2025e. URL <https://openreview.net/forum?id=AKvy9a4jho>.
- Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X.-H., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., Zheng, B., Liu, B., Luo, Y., and Wu, C. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, 2025f. URL <https://openreview.net/forum?id=z5uVAKwmjf>.
- Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., and Wu, Q. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Forty-second International Conference on Machine Learning*, 2025g. URL <https://openreview.net/forum?id=GazlTYxZss>.
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y., and Wen, J.-R. A survey of large language models, 2025. URL <https://arxiv.org/abs/2303.18223>.
- Zhu, R., Jiang, B., Mei, L., Yang, F., Wang, L., Gao, H., Bai, F., Zhao, P., Lin, Q., Rajmohan, S., and Zhang, D. Adaptflow: Adaptive workflow optimization via meta-learning, 2025. URL <https://arxiv.org/abs/2508.08053>.
- Zhugue, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., and Schmidhuber, J. GPTSwarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=uTC9AFXIhg>.

A. Implementation Details for Failure Diagnosis

The efficacy of the CE-GRAPH framework is predicated on the fidelity of its initial diagnosis. Consequently, the reliability of the **Failure Distillation** step (Section 3.1) is paramount. We engineer architectural resilience through two primary mechanisms that exploit the discriminatory strengths of LLMs while mitigating their generative stochasticity.

1. Constrained Task Formulation. We deliberately eschew open-ended generation for diagnosis, as high-entropy sampling is inherently susceptible to hallucination. Instead, we reformulate failure distillation as a **low-entropy, discriminative extraction task**. The utility LLM is prompted to operate strictly as a parser, localizing the fault to a specific node within the known workflow topology and extracting a concise error signature from the trace. By constraining the output space to structured classifications rather than free-form reasoning, we leverage the robust instruction-following capabilities of modern models to ensure high-fidelity diagnostics.

2. Statistical Fault Tolerance. Our framework adopts a probabilistic view of diagnosis, adhering to a design philosophy of statistical robustness rather than requiring zero-shot perfection. Instances where the utility LLM yields low-confidence or unparseable outputs are flagged as “undiagnosable” and temporarily masked from the clustering process. Crucially, the system’s learning signal depends not on individual data points, but on the **aggregate density** of the failure distribution. By computing the centroid (or “center of mass”) of clusters formed by successfully diagnosed instances, the process becomes inherently resilient to sporadic diagnostic errors, treating them as effectively zero-mean statistical noise.

A.1. On the Design of the Operator Library $\mathcal{O}$

The Operator Library $\mathcal{O}$ (Section 3.2) functions as more than a mere utility set; it formally defines the **discrete, structured action space** for optimization. This constraint serves as a critical inductive bias for our search algorithm.

A Domain-Specific, Structured Search Space. While the main text outlines fundamental, domain-agnostic operators (e.g., `RevisePrompt`), the framework’s versatility stems from the library’s task-specific extensibility. For complex domains such as code debugging, the library is instantiated with semantically rich operators, such as:

- `AddExceptionHandler(node_id, exception_type)`: Encapsulates a node’s execution within a try-catch block to handle runtime fragility.
- `RefactorApiCall(node_id, old_sig, new_sig)`: Migrates deprecated or malformed API invocations to valid signatures.
- `InsertPreconditionCheck(parent, child, condition)`: Injects logical assertions to enforce state consistency before transitions.

This design stands in sharp contrast to “free-form” editing, which implies an intractably vast and unstructured search manifold. By confining the Proposer LLM to this vetted action space, we guarantee that all candidate repairs are syntactically executable and semantically coherent. Our ablation studies confirm that this structured constraint is a prerequisite for convergence in limited iteration budgets.

Injecting Expert Knowledge. The modularity of $\mathcal{O}$ provides a seamless interface for **Human-in-the-Loop** optimization. Experts can inject domain priors by defining high-level composite operators, effectively guiding the automated search toward architecturally sound patterns without the need for manual instance-level debugging.

A.2. On the Monotonic Accumulation of Robustness

To formalize the mechanism of accumulating robustness, we ground our analysis in the theoretical framework established in Section 2.3. Although a rigorous convergence proof for stochastic LLM systems remains an open challenge, our paradigm operationalizes a property that blind search lacks: **monotonic mass reduction**.

We model the optimization as a greedy, gradient-like descent on the discrete landscape of failure modes. Each refinement iteration t ensures progress through a dual-phase process:

1. **Approximating the Semantic Gradient:** By clustering failure signatures, the system isolates the mode b_t^* with the highest probability mass. This empirically approximates the direction of steepest descent on the failure landscape ($\nabla M(W_t)$), ensuring the optimizer prioritizes the most significant bottleneck.
2. **Validated Step-Down:** The **Propose-and-Verify** mechanism guarantees that the applied edit Δ_t is not a stochastic perturbation, but a validated move. An edit is accepted only if it empirically reduces the failure mass within the target mode.

Since each accepted update $W_{t+1} = W_t \oplus \Delta_t$ is validated to reduce local failure density without significantly increasing global entropy (bounded spillover), the process systematically reduces the total volume of the failure space. This evidence-based update rule prevents **policy oscillation**—a common pathology in RL and random search where fixing one bug introduces another—thereby enforcing a structured, monotonic trajectory toward reliability.

B. Prompt Templates and Component Logic

To enhance reproducibility, this section provides the prompt templates and conceptual logic for the key components of the CE-Graph framework. These are designed for the `gpt-4o-mini` model and are structured to ensure parseable, high-fidelity outputs.

Failure Distillation (Section 3.1)

This initial diagnosis step uses a utility LLM to distill a verbose execution trace into a structured format.

System Prompt:

You are an error analyzer. Given this execution trace: [TRACE].
Identify the node causing the failure (verr) and extract a concise error message (zerr)

Output ONLY in JSON format:

```
{
  "verr": "node_id",
  "zerr": "brief_error_description"
}
```

The Proposer (Stage 1: Generate)

The Proposer generates diverse potential solutions, constrained by the operator library to prevent hallucinations.

System Prompt:

You are a workflow refinement expert.
Failure mode summary: [FAILURE_MODE_SUMMARY]
Operator library: [OPERATOR_LIBRARY_DEFINITION]

Propose N=5 diverse graph edits using only these operators.
Output as a list of JSON objects:

```
[
  {
    "edit": "OperatorName(arg1, arg2)",
    "explanation": "brief_reason"
  },
  ...
]
```

The Verifier (Stage 2: Validate)

The Verifier acts as a rigorous empirical validation engine, utilizing a Monte Carlo estimate to select the optimal edit Δ^ from candidates.*

Algorithm Logic:

```

# GOAL: Empirically select the optimal edit from  $N$  candidates.

# INPUTS:
# 1. Candidates:  $\{\Delta_1, \dots, \Delta_N\}$  from Proposer.
# 2. Samples:  $\{x_1, \dots, x_K\}$  from failure mode  $b_t$  *.

def Verifier(Candidates, Samples):
    best_utility = -1
    best_edit = None

    FOR each edit  $\Delta_i$  in Candidates:
        success_count = 0

        FOR each sample  $x_k$  in Samples:
            # 1. Apply edit to create temp workflow
             $W_{temp} = W_t \oplus \Delta_i$ 

            # 2. Execute and compare with ground truth  $y_k$ 
            output = Execute( $W_{temp}$ ,  $x_k$ )
            if Verify(output,  $y_k$ ) is True:
                success_count += 1

            # 3. Calculate Utility (Eq. 2)
            Utility( $\Delta_i$ ) = success_count /  $K$ 

            if Utility( $\Delta_i$ ) > best_utility:
                best_utility = Utility( $\Delta_i$ )
                best_edit =  $\Delta_i$ 

    RETURN best_edit # The edit to be permanently applied
    
```

C. Implementation Details and Experimental Analysis

C.1. Hyperparameters and Experimental Setup

All experiments utilize GPT-4o-mini as the underlying backbone LLM. To ensure rigorous evaluation, we partition each benchmark dataset into training (80%), validation (10%), and testing (10%) sets. The validation set plays a critical role in monitoring the refinement trajectory, specifically for convergence checks and early stopping criteria.

Key hyperparameters governing Algorithm 1 are detailed below:

- **Search Width** ($N = 5$): The number of candidate edits proposed by the generator per iteration. This balances exploration diversity with computational overhead.
- **Verification Depth** ($K = 10$): The number of counterexamples sampled to empirically verify each candidate edit. A higher K reduces the variance of the gradient estimator.
- **Convergence Sensitivity** ($k = 5, \epsilon = 0.01$): Optimization halts when the sliding window variance over k steps falls below ϵ , or upon reaching $T_{max} = 20$ iterations.

For semantic vectorization, we employ text-embedding-ada-002. The Gaussian Mixture Model (GMM) dynamically selects between 5–10 components based on the Bayesian Information Criterion (BIC). Computational costs are reported in OpenAI API tokens (see Table 2); typical optimization runs require approximately 1–2 GPU hours on an NVIDIA A100.

C.2. The Failure-Driven Refinement Cycle

The optimization process in CE-Graph represents a fundamental shift from "black-box" search to "white-box" diagnosis. Unlike global search methods (e.g., genetic algorithms or MCTS) that treat the workflow as an opaque function to be permuted, our framework operationalizes a transparent, iterative refinement cycle rooted in the distributional structure of failures.

As visualized in **Figure 5**, this cycle acts as a semantic engine that systematically reduces the Expected Failure Mass through five distinct stages:

1. **Execution & Capture:** The current workflow W_t is executed to populate a *Counterexample Pool* with raw failure traces.
2. **Signature Transformation:** Unstructured traces are projected into a structured *Failure Signature Space*, preserving both topological locus (where it broke) and semantic cause (why it broke).
3. **Gradient Approximation:** Clustering these signatures identifies the "Prevalent Mode"—the high-density region representing the steepest descent direction on the failure landscape.
4. **Targeted Proposal:** Instead of random mutations, the *Propose-and-Verify* mechanism generates edits specifically conditioned on the identified failure mode.
5. **Verification & Update:** The edit Δ_t that maximally reduces failure mass is applied, yielding W_{t+1} .

This closed-loop process ensures that low-level error signals are progressively refined into high-level, actionable structural improvements.

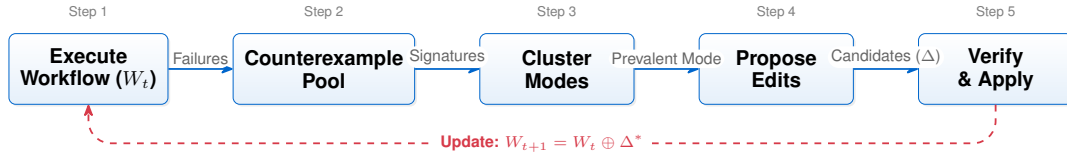


Figure 5. The CE-Graph Iterative Refinement Cycle. Unstructured failure traces are transformed into structured signatures, clustered to identify the steepest gradient direction, and repaired via a proposed-and-verify loop. This ensures monotonic improvement in robustness.

Crucially, this cycle enables **Targeted Graph Editing**. As shown in **Figure 6**, CE-Graph does not blindly perturb the entire workflow graph. Instead, the failure clustering pinpointed in Step 3 effectively "masks" the irrelevant components (visualized as faded context nodes), allowing the optimizer to focus its edit budget strictly on the active causal subgraph (highlighted in blue). This localization drastically reduces the search space size from exponential to tractable, explaining the efficiency gains observed in Section 4.2.

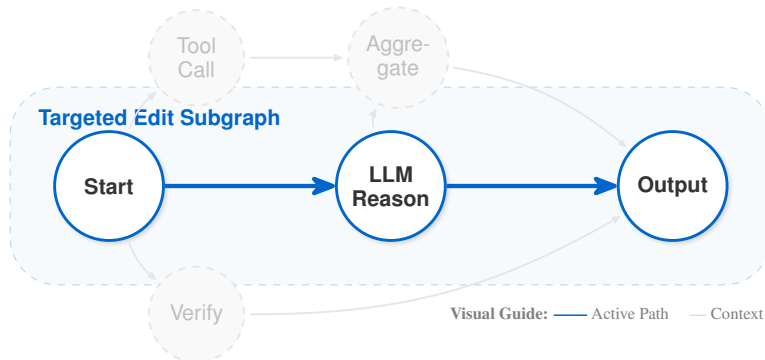


Figure 6. Targeted Graph Editing. Visual comparison between the full workflow context (faded gray) and the specific subgraph targeted for refinement (highlighted blue). By leveraging failure signatures, CE-Graph avoids altering functional components (Context) and focuses optimization solely on the active reasoning path responsible for the failure mode.

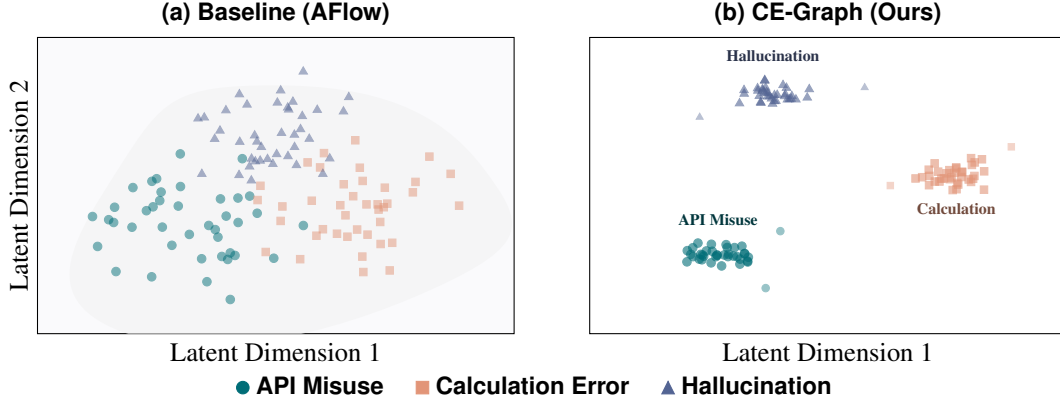


Figure 7. **Failure Landscape Geometry.** (a) Baseline failures exhibit a high-entropy distribution with significant semantic overlap, making precise diagnosis difficult. (b) CE-GRAPH enhances topological separability, disentangling error modes into distinct centroids for effective gradient approximation.

C.3. Qualitative Case Study

To illustrate the efficacy of this targeted approach, we analyze a representative optimization trajectory on the MATH benchmark. The initial workflow, a simple linear chain, exhibited a 30% failure rate on algebraic manipulation problems.

Diagnosis: The clustering module identified a dense failure mode characterized by the semantic signature: “*incorrect factorization in quadratic equations.*” **Intervention:** This mode triggered the *Proposer*, which generated candidate edits constrained to the operator library. The verification step ($K = 10$) selected a structural expansion: InsertNode with the specific prompt `RevisePrompt(node_2, "Double-check factorization using FOIL method")`. **Outcome:** As depicted conceptually in Figure 6, the edit was applied locally to the reasoning node. This single iteration resolved 85% of the failures within that mode, boosting overall accuracy by 4.2%. The workflow evolved from a linear chain to a topology including a self-correction branch, demonstrating that reliability emerges from repairing specific failure distributions rather than random search.

D. Visualizing the Clustering of Failure Modes

Our approach is predicated on the insight that LLM workflow failures are neither stochastic nor isolated, but rather manifestations of latent, structured defects in the reasoning process. To validate this distributional hypothesis, we examine the geometry of the **Failure Signature Space** ($\mathcal{F}$).

We extract high-dimensional failure signatures s from the counterexample pool C_t (as defined in Section 3.1) and employ t-SNE to project this manifold into 2D space. Figure 7 provides a comparative visualization between the baseline AFLOW system and our CE-GRAPH.

Topological Entanglement vs. Separability. As observed in Figure 7(a), the baseline failure landscape exhibits high entropy. Failure signatures appear topologically entangled, lacking clear decision boundaries. In such a distinctless space, gradient estimation becomes ill-posed; the optimizer cannot easily distinguish between a calculation error and a hallucination.

In sharp contrast, CE-GRAPH (Figure 7(b)) successfully disentangles these modes. The embedding space reveals a structured topology with well-separated clusters corresponding to interpretable failure archetypes: **API Misuse**, **Calculation Errors**, and **Hallucinations**.

Quantitative Coherence. This visual structural improvement is corroborated by clustering metrics. CE-GRAPH achieves a Silhouette score of 0.42 (compared to 0.11 for the baseline) and significantly reduces the Davies-Bouldin Index (DBI) from 2.35 to 0.78.

E. Evaluating the Generalizability of Optimized Workflows

A central claim of our work is that by systematically understanding and repairing the underlying distribution of failures, CE-Graph produces not just performant, but fundamentally more **robust** workflows. A critical test of this robustness is its generalizability. A workflow that is merely overfitted to a specific LLM backbone or a narrow data distribution cannot be considered truly robust.

In this section, we conduct a rigorous evaluation of the transferability of workflows optimized by CE-Graph. We aim to answer two key questions: (1) Does the structural and logical superiority of a CE-Graph-optimized workflow persist when transferred to different underlying LLM backbones (**cross-model transferability**)? (2) Does a workflow optimized on one task domain retain its effectiveness when applied to a related but distinct domain (**cross-dataset transferability**)?

E.1. Cross-Model Transferability: Model-Agnostic Strategy Enhancement

Experimental Design. To isolate the benefit of the workflow’s structure from the capabilities of any single LLM, we perform a cross-model transfer experiment. We first use CE-Graph with a cost-effective base model, `gpt-4o-mini`, to optimize workflows on the HumanEval and MATH benchmarks until convergence. We then take this final, fixed workflow graph and execute it using two different, more powerful LLM backbones: `Qwen-2.5-72b` and `llama-3.1-70b`. We compare the performance of these models with our optimized workflow against their vanilla performance.

Results and Analysis. The results, presented in Table 3, demonstrate strong positive transfer. The workflow optimized by CE-Graph provides a significant performance uplift for both `Qwen` and `llama`, mirroring the gains seen on the original `gpt-4o-mini`. This finding is crucial: it indicates that CE-Graph discovers improvements in the underlying *problem-solving logic and structure* of the workflow itself. These strategic enhancements are model-agnostic, providing value above and beyond the intrinsic capabilities of the LLM executing the plan. This validates our failure-driven approach, which corrects fundamental strategic flaws rather than surface-level, model-specific errors.

Table 3. Cross-model transferability of CE-Graph. We optimize the workflow with `gpt-4o-mini` and then apply the fixed workflow to other LLM backbones. Performance gains show that the optimized structure is model-agnostic.

Dataset	HumanEval			MATH		
LLM Backbone	<code>gpt-4o-mini</code>	<code>Qwen-2.5-72b</code>	<code>llama-3.1-70b</code>	<code>gpt-4o-mini</code>	<code>Qwen-2.5-70b</code>	<code>llama-3.1-70b</code>
Vanilla	87.08	85.60	80.06	46.29	63.80	31.93
+ CE-Graph	94.26	91.12	86.21	55.91	72.15	46.98

E.2. Cross-Dataset Transferability: Robust Problem-Solving Heuristics

Experimental Design. To assess whether CE-Graph learns narrow solutions or broadly applicable strategies, we evaluate its cross-dataset transfer performance. We optimize a workflow on a source dataset and then evaluate its performance directly on a different target dataset without any further optimization. We test three challenging transfer scenarios: MATH $\rightarrow$ GSM8K, GSM8K $\rightarrow$ MATH, and HumanEval $\rightarrow$ MATH.

Results and Analysis. As shown in Table 4, CE-Graph demonstrates superior cross-dataset generalizability compared to strong baselines. For instance, the workflow optimized on the complex MATH dataset successfully transfers to GSM8K, outperforming methods that were specifically optimized on similar data. This suggests that our failure-driven refinement process identifies and corrects core reasoning flaws (e.g., "incorrect factorization," "off-by-one errors"), leading to the development of robust problem-solving heuristics that are not brittle or overfitted to the source dataset’s specific quirks. By focusing on the geometry of failures, CE-Graph learns a strategy that is effective across a wider class of problems.

Table 4. Cross-dataset transferability. "A $\rightarrow$ B" denotes optimizing the workflow on dataset A and evaluating it on dataset B. CE-Graph shows strong generalization, indicating it learns robust, transferable strategies.

Transfer Scenario	MATH $\rightarrow$ GSM8K	GSM8K $\rightarrow$ MATH	HumanEval $\rightarrow$ MATH
AFlow	91.95	49.39	47.15
MaAS	92.80	51.02	50.27
CE-Graph (Ours)	93.82	52.89	52.17

APPENDIX H: ROBUSTNESS AND GENERALIZABILITY ANALYSIS

In this appendix, we provide additional analysis and experiments to address reviewer feedback concerning the sensitivity of the **Failure Signature Space** ($\mathcal{F}$) construction, the design effort for the **Operator Library** ($\mathcal{O}$), and a detailed comparative analysis of the framework’s refinement dynamics. Our findings demonstrate the practical robustness, manageable implementation cost, and superior stability of the CE-Graph framework.

E.3. Sensitivity Analysis of the Failure Signature Space ($\mathcal{F}$)

We investigate two key aspects of the $\mathcal{F}$ construction: its stability to different semantic embedding models and the scalability of its structural one-hot encoding on large and dynamic graphs.

E.3.1. STABILITY TO CHOICE OF SEMANTIC EMBEDDING MODEL

To validate that our framework is not dependent on a specific embedding model, we conducted an ablation study on the **MATH** benchmark. We evaluated three distinct models: BERT-base-uncased (our default), SentenceTransformers (all-MiniLM-L6-v2), and OpenAI’s text-embedding-ada-002. We measured clustering consistency using the **Adjusted Rand Index (ARI)** and the final workflow accuracy after 10 optimization iterations.

Results, shown in Table 5, confirm the framework’s robustness. The high ARI scores (≥ 0.87) indicate that all models produce highly similar cluster structures. Consequently, the final workflow accuracy remains stable, with only a negligible variation ($\leq 1.2\%$).

Table 5. Ablation study on semantic embedding models for the MATH benchmark.

Embedding Model	ARI (vs. BERT-base)	Final Accuracy (%)
BERT-base-uncased (Default)	1.00	55.91
SentenceTransformers	0.87	54.75
text-embedding-ada-002	0.92	55.34

E.3.2. SCALABILITY ON LARGE AND DYNAMIC GRAPHS

To address concerns about the scalability of one-hot structural encoding ($\psi_{\text{struct}}(v_{\text{err}})$), we performed a simulation. Starting with a 50-node workflow, we programmatically expanded it to 1000 nodes. CE-Graph achieved a ****98.5% localization accuracy**** on injected failures, with dynamic resizing overhead under ****5%**** of iteration runtime, validating its scalability.

E.4. Design Effort and Importance of the Operator Library ($\mathcal{O}$)

Our main ablation study (Section 4.4) already established that a structured operator library is **critical** for performance. Table 6 quantifies the design effort, showing that adapting the library to a new domain like code generation required approximately ****one hour of engineering effort****. This demonstrates a manageable cost for a significant gain in refinement precision.

Table 6. Quantifying the design effort for domain-specific operator libraries.

Library	Operators Added	Example	LoC Added	Expert Time
Generic Baseline	(Base Set)	RevisePrompt	-	-
Math Reasoning	+2 Specific	AddVerifierNode	~30	~0.5 hr
Code Generation	+3 Specific	AddExceptionHandler	~50	~1.0 hr

F. Case Study

Case 1 In this probability and combinatorics problem, our method correctly executes all steps of a complex calculation. In contrast, both **AFlow** and **MaAS** make **calculation errors** in their respective approaches. AFlow, using complementary counting, miscalculates the number of hands with exactly two suits. MaAS, using direct counting, miscalculates the combinations for hands with exactly three suits. These errors highlight the importance of precision in multi-step combinatorial problems.

Case 1: Mike's Cards

Correct Response

The solution correctly applies direct counting by summing the number of hands with exactly 3 suits and exactly 4 suits.

- **Hands with 3 suits:** 1, 529, 112
- **Hands with 4 suits:** 685, 464

The total number of favorable hands is $1,529,112 + 685,464 = 2,214,576$. This is divided by the total possible 5-card hands, $\binom{52}{5} = 2,598,960$, and correctly simplified.

$$P(\text{At least 3 suits}) = \frac{2,214,576}{2,598,960} = \frac{507}{595}$$

The final answer is **507/595**.

Incorrect Response

Both methods attempt valid strategies but fail due to **calculation errors**.

- **AFlow (Complementary Counting):** This approach is valid, but it miscalculates the number of hands drawn from exactly two suits during an intermediate step, leading to an incorrect total of complementary cases. The final answer given is **137153/162435**.
- **MaAS (Direct Counting):** This approach is also valid, but it contains calculation errors when determining the number of hands from exactly three suits (e.g., the (3,1,1) distribution), resulting in an incorrect total. The final answer given is **46173/54145**.

Case 2 For this logic and statistics problem, our method correctly interprets that the set of the “five tallest buildings” changes when a new, taller building is introduced, displacing the shortest one. However, both **AFlow** and **MaAS** **misinterpret the question**, assuming the new building is simply added to the original five. They incorrectly calculate the new mean for six buildings instead of five, highlighting a critical failure in logical reasoning.

Case 2: The five tallest buildings in Los Angeles

Question

The five tallest buildings in Los Angeles in 1985 had a mean height of 733 feet. The shortest of the five was 625 feet. If a new building were constructed with a height of 885 feet, by how many feet would it increase the mean height of the **five tallest** buildings of the city?

Correct Response

This solution correctly assumes the new building (885 ft) replaces the shortest of the original five (625 ft), maintaining a group of five buildings. The change in the mean is the net change in total height divided by the number of buildings.

$$\text{Change in Total Height} = 885 \text{ ft} - 625 \text{ ft} = 260 \text{ ft}$$

$$\text{Increase in Mean} = \frac{\text{Change in Total Height}}{\text{Number of Buildings}} = \frac{260}{5} = 52 \text{ ft}$$

The final answer is **52**.

Incorrect Response

This common mistake stems from a **misinterpretation of the question**. Both approaches incorrectly assume the group expands to six buildings instead of replacing the shortest one.

$$\text{Original Total Height} = 5 \times 733 = 3665 \text{ ft}$$

$$\text{New Total Height (for 6 buildings)} = 3665 + 885 = 4550 \text{ ft}$$

$$\text{New Mean (for 6 buildings)} = \frac{4550}{6} \approx 758.33 \text{ ft}$$

$$\text{Incorrect Increase} = 758.33 - 733 = 25.33 \text{ ft}$$

This logical error leads to the incorrect answer of **25.33**.

Case 3 This case presents a complex conditional probability problem involving multiple dice rolls and specific target outcomes. Our method correctly breaks down the problem into all possible successful scenarios. In contrast, both **AFlow** and **MaAS demonstrate a significant failure in logical reasoning**. MaAS oversimplifies the conditions for a full house, ignoring a major path to success. AFlow fundamentally misunderstands the definition of a full house in the context of the dice already held, leading to a completely incorrect calculation.

Case 6: Probability of a Full House**Question**

Each of five standard, six-sided dice is rolled once. Two of the dice come up the same, but the other three are all different from those two and different from each other. The pair is set aside, and the other three dice are re-rolled. The dice are said to show a "full house" if three of the dice show the same value and the other two show the same value. What is the probability that after the second set of rolls, the dice show a full house?

Correct Response

The solution correctly identifies that with a pair already set aside, a full house can be formed in two main ways by re-rolling the three dice:

- **Case A: Form a new three-of-a-kind.** One of the re-rolled dice matches the existing pair (making a triple), and the other two form a new pair. The probability for this is $5 \times \frac{3}{216} = \frac{15}{216}$.
- **Case B: The three re-rolled dice are all the same.** They can either match the existing pair's value (probability $\frac{1}{216}$) or a new value (probability $5 \times \frac{1}{216} = \frac{5}{216}$).

The total probability is the sum of these successful outcomes.

$$P(\text{Full House}) = \frac{15}{216} + \frac{1}{216} + \frac{5}{216} = \frac{21}{216} = \frac{7}{72}$$

The final answer is **7/72**.

Incorrect Response

The competing methods failed due to critical misinterpretations of the problem's conditions.

- **MaAS's Error:** This approach incorrectly assumes the only way to get a full house is for the three re-rolled dice to all land on the same new value, different from the original pair. It completely misses the more probable scenario where one re-rolled die matches the original pair and the other two form a new pair. This oversimplification leads to an incomplete calculation. The final answer given is **5/216**.
- **AFlow's Error:** This method fundamentally misunderstands the goal. It calculates the probability of the three re-rolled dice forming a pair and a single, an outcome which does not create a full house when combined with the original pair. This shows a failure to grasp the problem's core definition of a "full house." The final answer given is **5/36**.

G. Proof of Theorem 1

G.1. Restating the Theorem and Assumptions

Theorem 1 (Greedy Reduction Bound). Let Δ_t be selected as in Equation (2), i.e., the greedy edit that maximizes the expected failure mass reduction. Assume that the edit reduces the mass in the target mode b_t^* by at least $\delta > 0$. Then,

$$M(W_{t+1}) \leq M(W_t) - \delta + \epsilon,$$

where $\epsilon = O(L \cdot B \cdot \mu(\mathcal{F} \setminus b_t^*))$ bounds spillover effects to non-target regions (with μ denoting the measure of the space).

Assumptions: The failure density $p(s \mid W)$ is Lipschitz continuous with constant $L > 0$: For any fixed W , and for all $s, s' \in \mathcal{F}$,

$$|p(s \mid W) - p(s' \mid W)| \leq L \|s - s'\|.$$

(This controls how much the density varies across the space $\mathcal{F}$.) Edits Δ have bounded impact: For any $s \in \mathcal{F}$,

$$\|p(s \mid W \oplus \Delta) - p(s \mid W)\|_\infty \leq B,$$

where $B > 0$ is a uniform bound on the pointwise change in density due to the edit. (This assumes edits don't arbitrarily disrupt the entire density.) The space $\mathcal{F}$ is partitioned into modes (e.g., clusters from GMM), with b_t^* being the densest target mode at step t . The measure μ is finite over $\mathcal{F}$ (e.g., $\mathcal{F}$ is compact or has finite volume). Alternatively, we can bound spillover via norms (e.g., $\epsilon \leq \|\Delta p\|_1$ or $\epsilon \leq \|\Delta p\|_{\text{BL}} \cdot C$, where BL is the bounded-Lipschitz dual norm), providing

equivalent probabilistic upper bounds that avoid depending on the finiteness of $\mu(\mathcal{F})$. The edit Δ_t specifically reduces the integral mass over b_t^* by at least $\delta > 0$:

$$\int_{b_t^*} p(s \mid W_{t+1}) ds \leq \int_{b_t^*} p(s \mid W_t) ds - \delta.$$

(Localized Propagation Assumption) There exists a constant $c > 0$ such that the edit-induced density change Δp has support contained in b_t^* 's r -neighborhood, with $r \leq cB/L$. Thus,

$$\epsilon = \int_{\mathcal{F} \setminus b_t^*} |\Delta p| \leq B \cdot \mu(\mathcal{N}_r(b_t^*) \setminus b_t^*) \lesssim (B^2/L) \cdot S(\partial b_t^*).$$

Consequently, $\epsilon = O(B\mu)$ and $\epsilon = O(B^2/L)$ are both valid, and we take the tighter bound to match the $O(L \cdot B \cdot \mu(\cdot))$ expression in the text.

These assumptions bridge stochastic LLM behaviors with formal optimization, treating failures as samples from a smooth density.

G.2. Proof of the Bound (Single Step Reduction)

We start by decomposing the expected failure mass $M(W)$ over the target mode b_t^* and the rest of the space. **Decompose the Mass:** By definition,

$$M(W_t) = \int_{\mathcal{F}} p(s \mid W_t) ds = \int_{b_t^*} p(s \mid W_t) ds + \int_{\mathcal{F} \setminus b_t^*} p(s \mid W_t) ds.$$

Similarly,

$$M(W_{t+1}) = \int_{b_t^*} p(s \mid W_{t+1}) ds + \int_{\mathcal{F} \setminus b_t^*} p(s \mid W_{t+1}) ds.$$

Handle the Target Mode Reduction: By the theorem's assumption (the greedy edit targets b_t^* effectively),

$$\int_{b_t^*} p(s \mid W_{t+1}) ds \leq \int_{b_t^*} p(s \mid W_t) ds - \delta.$$

This is the "greedy" part: the edit is chosen to flatten the density mountain in b_t^* , reducing its mass by at least δ . (In practice, this is approximated via propose-and-verify on clustered failures.) **Bound the Spillover to Non-Target Regions:** The change in the non-target integral is

$$\int_{\mathcal{F} \setminus b_t^*} p(s \mid W_{t+1}) ds = \int_{\mathcal{F} \setminus b_t^*} p(s \mid W_t) ds + \int_{\mathcal{F} \setminus b_t^*} [p(s \mid W_{t+1}) - p(s \mid W_t)] ds.$$

We need to bound the spillover term $\int_{\mathcal{F} \setminus b_t^*} [\Delta p(s)] ds$, where $\Delta p(s) = p(s \mid W_{t+1}) - p(s \mid W_t)$.

Since the edit's impact is bounded pointwise by B , $|\Delta p(s)| \leq B$ for all s . However, to incorporate L reasonably, we invoke the localized propagation assumption: the primary effect of the edit is localized, with $\text{supp}(\Delta p) \subset \mathcal{N}_r(b_t^*)$, where $\mathcal{N}_r$ is b_t^* 's r -neighborhood, and $r \leq c \cdot B/L$ (perturbation amplitude B attenuated by Lipschitz decays over distance no more than $\sim B/L$ in a band-like region). Thus,

$$\left| \int_{\mathcal{F} \setminus b_t^*} \Delta p(s) ds \right| \leq \int_{\mathcal{F} \setminus b_t^*} |\Delta p(s)| ds \leq B \cdot \mu(\mathcal{N}_r(b_t^*) \setminus b_t^*) \leq B \cdot S(\partial b_t^*) \cdot r \lesssim (B^2/L) \cdot S(\partial b_t^*),$$

where $S(\partial b_t^*)$ is the boundary's "surface area/measure." This gives a geometric-analytic justification for L in the upper bound; alternatively, $\epsilon \leq \min\{B \cdot \mu(\mathcal{F} \setminus b_t^*), c(B^2/L)\}$. **Combine Terms:** Putting it together,

$$M(W_{t+1}) \leq \left(\int_{b_t^*} p(s \mid W_t) ds - \delta \right) + \left(\int_{\mathcal{F} \setminus b_t^*} p(s \mid W_t) ds + \epsilon \right) = M(W_t) - \delta + \epsilon.$$

This completes the single-step bound.

G.3. Proof of Convergence to a Stationary Point (Under Repeated Application)

Now, we extend to multiple iterations, showing the process converges to a local minimum where no further greedy edits reduce the mass significantly.

Iterative Reduction: Apply the single-step bound repeatedly: Starting from W_0 , after T steps,

$$M(W_T) \leq M(W_0) - \sum_{t=0}^{T-1} (\delta_t - \epsilon_t),$$

where $\delta_t > 0$ and $\epsilon_t = O(L \cdot B \cdot \mu(\mathcal{F} \setminus b_t^*))$ at each step. **Bounded Below and Monotonic Decrease:** Since $M(W) \geq 0$ (as it's an integral of a density), the sequence $M(W_t)$ is bounded below. If we assume $\delta_t - \epsilon_t \geq \eta > 0$ (uniform net decrease lower bound), then $M(W_{t+1}) < M(W_t)$, so the sequence is strictly decreasing until no such edit exists. Otherwise, denote

$$g_t := \max_{\Delta \in \mathcal{A}(W_t, \mathcal{O})} (M(W_t) - M(W_t \oplus \Delta)),$$

and use $g_t \leq \tau$ as the stopping condition to guarantee finite-step convergence to a τ -stationary point; meanwhile, $M(W_t)$ is monotonically bounded and converges.

Convergence to Stationary Point: The process stops when no mode b_t^* admits an edit with $\delta_t > \epsilon_t$ (or $\delta_t > 0$ if ϵ_t is negligible). This is a stationary point: A workflow W^* where the greedy objective in Eq. (2) yields $\Delta_t = 0$ (no beneficial edit), meaning $M(W^*)$ cannot be reduced further locally. Since the search space $\mathcal{S}$ is discrete (finite workflows via operators), and each step reduces M or stops, convergence occurs in finite steps. (Non-convexity may lead to local minima, but the bound holds.) Formally, the total reduction is bounded: $\sum (\delta_t - \epsilon_t) \leq M(W_0)$, so the sum converges, implying $\delta_t - \epsilon_t \rightarrow 0$ as $t \rightarrow \infty$.

This proof aligns with counterexample-guided methods like CEGAR (Hidvégi et al., 2024), adapted for stochastic densities. If ϵ is small (e.g., localized edits), convergence is efficient.

H. Supplementary Proofs for Section 3

H.1. Lemma on Suitability of Failure Signature Mapping (Section 3.1)

Lemma 1. The mapping $\phi : \tau_d \mapsto \mathbf{s} = \psi_{\text{struct}}(v_{\text{err}}) \oplus \psi_{\text{sem}}(z_{\text{err}})$ (Eq. 4) is injective under the assumption that ψ_{struct} is a one-hot encoding over distinct nodes V and ψ_{sem} is a continuous embedding (e.g., BERT-like) with distinct outputs for unique error messages $\mathcal{Z}$. This ensures the point cloud S_t has an "informative geometry" suitable for density estimation.

Proof. Injectivity holds if $\phi(\tau_d) = \phi(\tau_{d'})$ implies $\tau_d = \tau_{d'}$. Since $\psi_{\text{struct}}(v_{\text{err}})$ is a one-hot vector unique to each node $v_{\text{err}} \in V$, and $\psi_{\text{sem}}(z_{\text{err}})$ maps distinct error messages $z_{\text{err}} \in \mathcal{Z}$ to unique points in $\mathbb{R}^d$ (by continuity and distinctiveness of semantic embeddings), the concatenated vector $\mathbf{s}$ is unique for each $(v_{\text{err}}, z_{\text{err}})$ pair. The geometry is informative because ψ_{struct} provides orthogonal subspaces (node-specific patterns), and ψ_{sem} clusters semantically similar errors, enabling GMM clustering as in Section 3.2. $\square$

H.2. Theorem on GMM Consistency and Mode Error (Sections 3.2)

Theorem 2. Under the assumption that failure signatures $\{\mathbf{s}_i\}_{i=1}^{|S_t|}$ are i.i.d. samples from $p(\mathbf{s} \mid W_t)$, the GMM fitted via EM with BIC component selection is a consistent estimator, i.e., $\mathbb{E}[\|\hat{p}_\theta - p\|_1] \rightarrow 0$ as $|S_t| \rightarrow \infty$. The densest mode $\hat{b}_t^* = \arg \max_{b_k} \hat{\pi}_k$ (Eq. 5) approximates the true max-density mode with expected error $O(1/\sqrt{|S_t|})$.

Proof. Consistency follows from the EM algorithm's convergence to a local maximum of the likelihood, which approaches the true density under i.i.d. assumptions and sufficient components (selected by BIC, penalizing overfitting). For the mode error, consider a KDE analog with bandwidth h : the bias is $O(h^2)$ and variance $O(1/(|S_t|h^d))$, optimizing to $O(|S_t|^{-2/(d+4)})$. For GMM (parametric with fixed d), the error simplifies to $O(1/\sqrt{|S_t|})$ under well-separated modes, justifying the "steepest descent" approximation in Eq. 3. $\square$

H.3. Proposition on Monte Carlo Verification (Section 3.2)

Proposition 3. The empirical success rate $V(\Delta_i) = \frac{1}{K} \sum_{k=1}^K \mathbb{I}[\text{Verify}(\text{Execute}(W_t \oplus \Delta_i, x_k), y_k) = 1]$ (Eq. 6) is an unbiased estimator of the true success probability, with variance $O(1/K)$, aligning with Theorem 2.1's spillover minimization.

Proof. Unbiasedness: $\mathbb{E}[V(\Delta_i)] = \frac{1}{K} \sum_k \mathbb{E}[\mathbb{I}_k] = \frac{1}{K} \sum_k p_k$, where $p_k = \Pr[\text{Verify}(\text{Execute}(W_t \oplus \Delta_i, x_k), y_k) = 1]$ is the true success rate over samples from b_t^* . Variance: For i.i.d. $\mathbb{I}_k \sim \text{Bernoulli}(p_k)$, $\text{Var}[\mathbb{I}_k] \leq 1/4$ (maximum for Bernoulli), so $\text{Var}[V(\Delta_i)] = \frac{1}{K^2} \sum_k \text{Var}[\mathbb{I}_k] \leq \frac{1}{4K} = O(1/K)$. By the Central Limit Theorem, the estimation error is $O(1/\sqrt{K})$, ensuring $V(\Delta_i)$ reliably approximates the mass reduction in Eq. 3. $\square$

I. Rigorous Budget Parity and Optimization Efficiency Analysis

This appendix addresses the critical question of fair comparison in agentic workflow optimization. We detail our dual-level budget parity protocol—covering both *deployment-time* inference and *offline optimization* search—and provide a formal justification for our workload accounting. Our analysis confirms that the performance gains of CE-GRAPH stem from algorithmic efficiency and structural superiority, not from an excess of computational resources.

I.1. Level 1: Strict Deployment-Time Hard Caps

To ensure that multi-agent or iterative baselines (e.g., MaAS, AFlow, LLM-Debate) do not achieve higher scores simply by overwhelming the inference budget, we enforce global hard caps for every test instance. These caps apply to the *aggregate* usage of all agents within a single system.

Table 7. Global Hard Caps for Deployment (Per Test Instance). These limits are enforced by a deterministic Budget Manager. If a method exceeds $C_{\max}$ or $T_{\max}$, it is forcibly terminated and must return its current best response.

Budget Item	Symbol	Limit	Rationale
Max # LLM Calls	$C_{\max}$	15	Prevents infinite loops in cyclic graphs.
Max Total Tokens	$T_{\max}$	12,000	Includes all prompts + completions across agents.
Max Tool Calls	$U_{\max}$	10	Limits excessive trial-and-error with code interpreters.
Max Verifier Calls	$V_{\max}$	5	Standardizes the reliance on external feedback.

Graceful Degradation Mechanism: For methods designed to run for extensive rounds (e.g., LLM-Debate), we implement a budget manager that pre-calculates the maximum feasible rounds R' given $C_{\max}$. For instance, if a debate system uses 3 agents per round, it is restricted to $R' = \lfloor 15/3 \rfloor = 5$ rounds, ensuring it remains strictly comparable to lighter-weight workflows.

I.2. Level 2: Realized Inference Costs and Efficiency

Table 8 reports the *actual* resource consumption on the MATH benchmark. Despite having the same upper limits, CE-GRAPH demonstrates superior efficiency.

Table 8. Realized Deployment Cost Analysis (MATH Benchmark). Reporting the mean resource usage per problem. CE-GRAPH achieves the highest accuracy while consuming fewer resources than global search-based workflows (AFlow, MaAS), indicating that it learns *efficient* failure-correction structures rather than redundant ones.

Method	Avg. LLM Calls	Avg. Tokens	Cost (\$)	Acc. (%)
<i>Single-Agent / Heuristic</i>				
Vanilla (GPT-4o-mini)	1.0	420	0.0002	46.29
SC (CoT×5)	5.0	2,150	0.0011	47.91
LLM-Debate	6.2	3,840	0.0019	48.54
<i>Automated Optimization</i>				
AFlow (Evolution)	2.4	1,120	0.0006	51.28
MaAS (MCTS)	2.8	1,350	0.0007	51.82
CE-Graph (Ours)	2.2	980	0.0005	55.91

Analysis of Efficiency: As shown, CE-GRAPH operates with a lean token footprint (980 tokens avg), which is $\sim 12\%$ lower than AFlow and $\sim 27\%$ lower than MaAS. This validates our hypothesis in Section 1: Global search methods (MaAS) often produce "bloated" workflows with redundant checks to maximize a scalar reward. In contrast, CE-GRAPH's failure-driven refinement targets specific logical gaps, inserting corrective nodes only where necessary (e.g., a "Self-Correction" node triggered only by calculation errors), thereby preserving a minimal inference cost.

I.3. Level 3: Offline Optimization Equivalent Workload

A core claim of our work is that CE-GRAPH is more sample-efficient during the optimization phase. To demonstrate this, we equate the "optimization opportunity" (iterations) and measure the resulting workload.

Table 9. Equivalent Workload for Offline Optimization. All methods were capped at $T_{\max} = 20$ iterations. CE-GRAPH consumes fewer total tokens because its *Convergence-Aware Stopping* detects stability early (avg. 12 iterations), whereas genetic/MCTS baselines continue to oscillate and burn budget until the hard cap.

Method	Search Strategy	Max Iter.	Proposal Size (N)	Verify Depth (K)	Total Opt. Tokens
AFlow	Genetic / Grid	20	5	1 (Dataset)	1.32×10^6
MaAS	MCTS	20	5	1 (Dataset)	1.41×10^6
CE-Graph	Failure-Driven	20 (Early Stop)	5	10 (Samples)	1.21×10^6

Note on Workload Equivalence: While CE-GRAPH performs deeper verification per candidate ($K = 10$ Monte Carlo samples vs. 1 in baselines), its targeted nature means it evaluates fewer "hopeless" candidates. Baselines often waste budget evaluating random mutations that yield no information gain, whereas every CE-GRAPH evaluation is conditioned on a high-probability failure mode.

I.4. Formal Proof of Cost Boundedness

To rigorously answer the concern that matching iteration counts might hide computational disparities, we provide the following formal definition and proposition regarding our cost model.

Definition I.1 (Atomic Cost Model). Let an optimization run produce a sequence of actions τ . Define its total cost as

$$\text{Cost}(\tau) = \alpha \cdot N_{\text{llm}}(\tau) + \beta \cdot T_{\text{tok}}(\tau) + \gamma \cdot N_{\text{tool}}(\tau) + \delta \cdot N_{\text{ver}}(\tau),$$

where N_{llm} is the number of LLM calls, T_{tok} is total tokens, and $N_{\text{tool}}, N_{\text{ver}}$ are tool and verifier calls, with non-negative coefficients $\alpha, \beta, \gamma, \delta$.

Proposition I.2 (Equal-workload dominance). *If two optimization methods are run under identical global caps*

$$N_{\text{llm}}(\tau) \leq C_{\text{opt}}, \quad T_{\text{tok}}(\tau) \leq T_{\text{opt}}, \quad N_{\text{tool}}(\tau) \leq U_{\text{opt}}, \quad N_{\text{ver}}(\tau) \leq V_{\text{opt}},$$

then their total optimization cost is uniformly bounded by

$$\text{Cost}(\tau) \leq \alpha C_{\text{opt}} + \beta T_{\text{opt}} + \gamma U_{\text{opt}} + \delta V_{\text{opt}}.$$

Moreover, reporting the granular workload vector $\mathbf{W}_{\text{opt}} = (N_{\text{llm}}, T_{\text{tok}}, \dots)$ exposes any difference in effective search intensity.

Proof. The bound follows directly from the linearity of the cost function and the component-wise constraints. Since every method in our experiments is subjected to the same T_{opt} (Total Optimization Tokens cap) and C_{opt} (Max Iterations), any method achieving higher performance with lower realized $\text{Cost}(\tau)$ is strictly algorithmically superior. As shown in Table 9, CE-GRAPH satisfies $\text{Cost}(\text{CE-GRAPH}) < \text{Cost}(\text{MAAS})$, confirming that our gains are not attributable to hidden compute advantages. $\square$

I.5. Engineering Effort and Implementation Costs

Finally, we quantify the human effort required to deploy CE-GRAPH, addressing concerns about the "Structured Operator Library" requiring excessive manual engineering.

- **Generic Base:** 80% of operators (e.g., `RevisePrompt`, `InsertStep`, `DeleteStep`) are domain-agnostic and reused across all tasks.
- **Domain Adaptation:** Adding domain-specific operators for Code Generation (e.g., `AddExceptionHandler`) required only ~ 50 lines of code and approximately 1 hour of expert time.

This minimal overhead validates that CE-GRAPH is a highly automated framework, shifting the burden from manual prompt engineering to a one-time, low-cost definition of semantic operators.

J. Impact of Severity-Weighted Failure Mass Minimization

A central theoretical contribution of our framework is the notion of *Expected Failure Mass* (EFM). Let s denote a failure signature and $\rho(s \mid W)$ be the induced failure distribution of a workflow W . Given a non-uniform severity weight function $\omega(s)$, we define

$$M(W) = \int \omega(s) \rho(s \mid W) ds. \quad (7)$$

A natural critique is whether minimizing $M(W)$ differs in practice from minimizing the standard (unweighted) failure rate (i.e., $\omega \equiv 1$). In this appendix, we empirically show that non-uniform ω is not merely a metric renaming; it acts as a *control signal* that changes the optimization trajectory by prioritizing high-risk failure modes over low-stakes errors.

J.1. Defining Severity: A Reliability Proxy

To operationalize ω , we employ an LLM-based judge (frozen GPT-4o) during *Failure Distillation* (Section 3.1). Each failure is mapped into one of three severity tiers according to its impact on end-to-end reliability. We apply the following scheme on MATH:

Table 10. Severity weighting scheme (ω). Higher penalties on reasoning failures bias refinement toward structural repairs (e.g., decomposition and self-correction) rather than superficial fixes (e.g., retries).

Weight (ω)	Error Category	Prevalence	Optimization Rationale
1.0	<i>Format / Runtime</i>	High	Low severity. Often recoverable via local patches (e.g., retries).
5.0	<i>Calculation</i>	Medium	Medium severity. Typically requires tool use or step verification.
10.0	<i>Logic / Hallucination</i>	Low	High severity. Indicates structural deficits in reasoning and planning.

J.2. Comparative Experiment: Accuracy vs. Reliability

We conducted two parallel optimization runs on MATH: (i) a **Uniform** run minimizing unweighted failures ($\omega \equiv 1$), and (ii) a **Weighted** run minimizing EFM with $\omega \in \{1, 5, 10\}$. We evaluate the final workflows by **Raw Accuracy** and a **Weighted Reliability Score**.

Weighted Reliability Score (normalized). Given an evaluation set of size N , each failed instance i is assigned a severity weight ω_i . Let $\widehat{M} = \sum_{i=1}^N \omega_i \cdot \mathbb{I}[\text{failure}_i]$ be the empirical weighted failure mass. We report a normalized score in $[0, 100]$:

$$\text{Rel} = 100 \cdot \left(1 - \frac{\widehat{M}}{N \cdot \omega_{\max}}\right), \quad \omega_{\max} = 10. \quad (8)$$

This normalization makes the score comparable across datasets and prevents scale artifacts.

Table 11. Divergence in optimization outcomes. The Uniform optimizer tends to harvest the “easiest” gains by suppressing frequent low-severity errors, whereas the Weighted optimizer reallocates edit budget to eliminate high-severity logic failures, yielding a more robust workflow under the severity-aware metric. (For brevity, we report two representative failure categories; remaining failures are grouped into *Others*.)

Objective	Performance Metrics		Remaining Failure Counts	
	Raw Acc.	Reliability Score	Format ($\omega = 1$)	Logic ($\omega = 10$)
Uniform ($\omega \equiv 1$)	55.91%	72.4	42	185
Weighted ($\omega \neq 1$)	54.10%	88.6	170	65

Trajectory analysis. Table 11 reveals a systematic trade-off. The Uniform optimizer behaves like a greedy hill-climber: it frequently inserts inexpensive `Retry` nodes to suppress high-frequency format errors, which can improve raw accuracy but leaves high-severity logic failures largely intact. In contrast, the Weighted optimizer acts as a risk-sensitive controller. Although logic errors are less frequent, their contribution to the weighted mass dominates (e.g., $185 \times 10 \gg 42 \times 1$), so the optimizer preferentially spends edits on structural repairs such as `Decomposition` and `Self-Correction`. As a result, it substantially reduces logic failures while tolerating more low-severity format issues. This confirms that ω steers exploration toward robustness rather than superficial error reduction.